

Sistemas de Equações Lineares – Métodos Diretos

Cálculo Numérico – Bacharelado em Ciência da Computação

Profa. Angela Leite Moreno

Universidade Federal de Alfenas

Conteúdo da Aula

- 1 Introdução
- 2 Número de Soluções
- 3 Existência e Unicidade
- 4 Métodos Diretos vs. Iterativos
- 5 Eliminação de Gauss
- 6 Estratégias de Pivoteamento
- 7 Custo Computacional
- 8 Fatoração LU
- 9 Matrizes com Estrutura Especial
- 10 Condicionamento
- 11 Comparação dos Métodos
- 12 Exercícios
- 13 Encerramento
- 14 Referências

Onde surgem os Sistemas Lineares?

Problema central

Dado $A \in \mathbb{R}^{m \times n}$ e $b \in \mathbb{R}^m$, encontrar $x \in \mathbb{R}^n$ tal que

$$Ax = b.$$

Aplicações do mundo real

- **PageRank** (Google): $\sim 10^{11}$ equações.
- **Regressão linear/ML**: equações normais.
- **Simulação física**: discretização de EDPs.
- **Redes elétricas**: lei de Kirchhoff.
- **Computação gráfica**: transformações.
- **Previsão do tempo**: sistemas dinâmicos.
- **Otimização de trânsito / metrô**: fluxo em redes.

Forma geral do sistema

$$\begin{cases} a_{11}x_1 + \cdots + a_{1n}x_n = b_1 \\ a_{21}x_1 + \cdots + a_{2n}x_n = b_2 \\ \vdots \\ a_{m1}x_1 + \cdots + a_{mn}x_n = b_m \end{cases}$$

Definição: Sistema Linear

Definição

Um **sistema linear** com m equações e n variáveis é escrito na forma

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n = b_2 \\ \vdots \\ a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n = b_m \end{cases}$$

em que:

- a_{ij} : **coeficientes**, com $1 \leq i \leq m$, $1 \leq j \leq n$;
- x_j : **variáveis**, $j = 1, \dots, n$;
- b_i : **constantes**, $i = 1, \dots, m$.

Notação Matricial

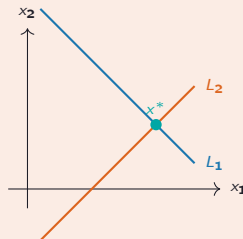
Representação compacta: $Ax = b$

$$A = \begin{pmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{m1} & \cdots & a_{mn} \end{pmatrix}, \quad x = \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix}, \quad b = \begin{pmatrix} b_1 \\ \vdots \\ b_m \end{pmatrix}$$

Objetivo

Encontrar x^* (vetor solução) tal que $Ax^* = b$.

Interpretação geométrica ($n = 2$)

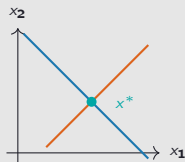


Cada equação é uma reta; a solução é o ponto de interseção.

Quantas Soluções Pode Ter um Sistema?

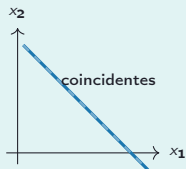
Para qualquer sistema linear $Ax = b$, **exatamente uma** das situações ocorre:

(i) Solução única



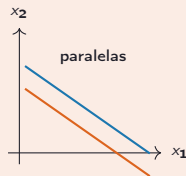
Retas **concorrentes**

(ii) Infinitas soluções



Retas **coincidentes**

(iii) Sem solução



Retas **paralelas**

Existência e Unicidade da Solução

Theorem (Existência e Unicidade)

$Ax = b$ tem solução **única** se, e somente se, $\det(A) \neq 0$ (i.e., A é não-singular).

Prova

($\Rightarrow$) Unicidade $\Rightarrow \det \neq 0$.

Suponha solução única x_0 . Se $\det(A) = 0$, existe $v \neq 0$ com $Av = 0$. Então $A(x_0 + v) = b$, contradição.

($\Leftarrow$) $\det \neq 0 \Rightarrow$ Unicidade.

A invertível $\Rightarrow x = A^{-1}b$. Se $Ax_1 = Ax_2 = b$, então $x_1 = A^{-1}b = x_2$. ■

Atenção prática

Nunca calcule $\det(A)$ para verificar singularidade!

Os algoritmos de fatoração detectam isso automaticamente com custo $O(n^3)$.

Duas Abordagens para Resolver $Ax = b$

Métodos Diretos

- Fornecem a solução **exata** (salvo erros de arredondamento)
- Terminam após um número **finito** de operações
- Exemplos: Gauss, Fatoração LU

Custo: $\mathcal{O}\left(\frac{2n^3}{3}\right)$ operações

Métodos Iterativos (Indiretos)

- Geram uma **sequência** $\{x^{(k)}\}$ que converge para x^*
- Partem de uma aproximação inicial $x^{(0)}$
- Exemplos: Gauss-Jacobi, Gauss-Seidel

Custo por iteração: $\mathcal{O}(n^2)$

Eliminação de Gauss — Ideia Central

Objetivo

Transformar o sistema $Ax = b$ em um sistema **equivalente** com matriz dos coeficientes **triangular superior** U , de resolução imediata.

Três etapas:

0. Montar a **matriz sanduíche** $[A | b]$
1. **Eliminação** progressiva (zerando abaixo da diagonal)
- F. **Substituição** retroativa (de baixo para cima)

Operações elementares permitidas

- Trocar duas equações: $L_i \leftrightarrow L_j$
- Multiplicar por escalar $\neq 0$: $L_i \leftarrow \alpha L_i$
- Somar múltiplo de outra linha:
$$L_i \leftarrow L_i - m_{ik} L_k$$

Sistemas obtidos por essas operações são **equivalentes**.

Etapa 0 — Matriz Sanduíche $[A | b]$

Sistema com 5 variáveis:

$$\begin{cases} \bullet x_1 + \bullet x_2 + \bullet x_3 + \bullet x_4 + \bullet x_5 = \bullet \\ \bullet x_1 + \bullet x_2 + \bullet x_3 + \bullet x_4 + \bullet x_5 = \bullet \\ \vdots \\ \bullet x_1 + \bullet x_2 + \bullet x_3 + \bullet x_4 + \bullet x_5 = \bullet \end{cases}$$

Representado como a **matriz aumentada**:

$$\left(\begin{array}{ccccc|c} \bullet & \bullet & \bullet & \bullet & \bullet & \bullet \\ \bullet & \bullet & \bullet & \bullet & \bullet & \bullet \\ \bullet & \bullet & \bullet & \bullet & \bullet & \bullet \\ \bullet & \bullet & \bullet & \bullet & \bullet & \bullet \\ \bullet & \bullet & \bullet & \bullet & \bullet & \bullet \end{array} \right)$$

Diagonal principal = posição dos *pivôs*



pivôs

Os **pivôs** a_{kk} controlam cada etapa da eliminação.

Etapa de Eliminação — Zerando a Coluna k

Fórmula do multiplicador

Para zerar o elemento a_{ik} (linha i , coluna k):

$$m_{ik} = \frac{a_{ik}^{(k-1)}}{a_{kk}^{(k-1)}}, \quad L_i \leftarrow L_i - m_{ik} L_k$$



Etapa Final — Substituição Retroativa

Sistema triangular superior resultante:

$$\left\{ \begin{array}{l} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1 \\ \quad a_{22}x_2 + \cdots + a_{2n}x_n = b_2 \\ \quad \quad \vdots \\ \quad \quad \quad a_{nn}x_n = b_n \end{array} \right.$$

Resolução de baixo para cima

$$\begin{aligned} x_n &= \frac{b_n}{a_{nn}} \\ x_{n-1} &= \frac{b_{n-1} - a_{n-1,n}x_n}{a_{n-1,n-1}} \\ &\vdots \\ x_i &= \frac{b_i - \sum_{j=i+1}^n a_{ij}x_j}{a_{ii}} \end{aligned}$$

Eliminação de Gauss: Resumindo

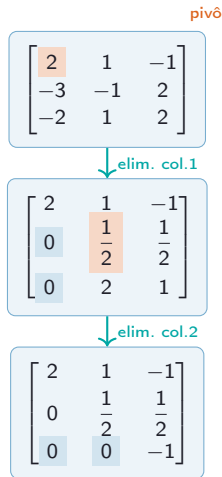
Objetivo: transformar $[A \mid b]$ em $[U \mid c]$ triangular superior via operações elementares que *preservam* as soluções.

Operações elementares (preservam soluções)

1. **Troca:** $L_i \leftrightarrow L_j$
2. **Escalonamento:** $L_i \leftarrow \lambda L_i, \lambda \neq 0$
3. **Combinação:** $L_i \leftarrow L_i + \lambda L_j$

Etapas da eliminação

Calcula multiplicadores $m_{ik} = \frac{a_{ik}^{(k-1)}}{a_{kk}^{(k-1)}}$ e aplica
 $L_i \leftarrow L_i - m_{ik} L_k$ para $i > k$.



Exemplo — Eliminação de Gauss

Sistema

$$3x_1 + 2x_2 + 4x_3 = 1; \quad x_1 + x_2 + 2x_3 = 2; \quad 4x_1 + 3x_2 - 2x_3 = 3$$

Matriz inicial:

$$\left(\begin{array}{ccc|c} 3 & 2 & 4 & 1 \\ 1 & 1 & 2 & 2 \\ 4 & 3 & -2 & 3 \end{array} \right)$$

Após etapa 1
($m_{21} = \frac{1}{3}$, $m_{31} = \frac{4}{3}$):

$$\left(\begin{array}{ccc|c} 3 & 2 & 4 & 1 \\ 0 & \frac{1}{3} & \frac{2}{3} & \frac{5}{3} \\ 0 & \frac{1}{3} & -\frac{10}{3} & \frac{5}{3} \end{array} \right)$$

Após etapa 2
($m_{32} = 1$):

$$\left(\begin{array}{ccc|c} 3 & 2 & 4 & 1 \\ 0 & \frac{1}{3} & \frac{2}{3} & \frac{5}{3} \\ 0 & 0 & -4 & 0 \end{array} \right)$$

Solução (substituição retroativa)

$$x_3 = 0; \quad x_2 = 5; \quad x_1 = -3$$

Eliminação de Gauss — Implementação Python

Triangularização

```
1 def gauss(A, b):
2     A = np.array(A, dtype=float)
3     b = np.array(b, dtype=float)
4     n = len(b)
5
6     # Eliminação progressiva
7     for k in range(n - 1):
8         # Pivoteamento parcial
9         p = np.argmax(abs(A[k:, k])) + k
10        A[[k, p]] = A[[p, k]]
11        b[[k, p]] = b[[p, k]]
12
13        for i in range(k + 1, n):
14            m = A[i, k] / A[k, k]
15            A[i, k:] -= m * A[k, k:]
16            b[i] -= m * b[k]
17
18    return A, b
```

Substituição retroativa + uso

```
1 def subst_retro(A, b):
2     n = len(b)
3     x = np.zeros(n)
4     for i in range(n - 1, -1, -1):
5         x[i] = (b[i]
6                 - A[i, i+1:] @ x[i+1:])
7                 / A[i, i]
8     return x
9
10 # --- Exemplo ---
11 import numpy as np
12 A = [[3, 2, 4],
13       [1, 1, 2],
14       [4, 3, -2]]
15 b = [1, 2, 3]
16
17 Au, bu = gauss(A, b)
18 x = subst_retro(Au, bu)
19 print(x) # [-3.  5.  0.]
```

Pivoteamento parcial

Troca a linha k pela de maior $|a_{ik}|$ abaixo — evita divisão por zero e reduz erros de arredondamento.

Custo

Eliminação: $\mathcal{O}(2n^3/3)$ flops
Substituição: $\mathcal{O}(n^2)$ flops

O Problema com o Pivô

Dois problemas críticos

- Se $a_{kk}^{(k-1)} = 0$: **divisão por zero** — método falha.
- Se $a_{kk}^{(k-1)} \approx 0$: **erros de arredondamento** — resultado impreciso.

Solução

Adotar uma **estratégia de pivoteamento**: escolher a melhor linha (ou coluna) pivotal antes de cada etapa.

Exemplo: pivô pequeno gera erro

Sistema com solução exata $\left(\frac{1}{3}, \frac{2}{3}\right)^T$:

$$\begin{cases} 0,0003 x_1 + 3 x_2 = 2,0001 \\ x_1 + x_2 = 1 \end{cases}$$

Com 4 dígitos e **sem** pivoteamento: erro de 100% em x_1 !

Com **pivoteamento parcial**: erro menor que 0,01%.

Pivoteamento Parcial

Estratégia

No início de cada etapa k ($k = 1, \dots, n - 1$):

1. Encontrar o elemento de **maior módulo** na coluna k , entre as linhas $k, k + 1, \dots, n$
2. Trocar as linhas k e p (se necessário), onde $|a_{pk}^{(k-1)}| = \max_{i \geq k} |a_{ik}^{(k-1)}|$

Exemplo visual — etapa 2 de uma matriz 4×4

$$A^{(1)} | b^{(1)} = \begin{pmatrix} 3 & 2 & 1 & -1 & 5 \\ 0 & 1 & 0 & 3 & 6 \\ 0 & -3 & -5 & 7 & 7 \\ 0 & 2 & 4 & 0 & 15 \end{pmatrix}$$

O elemento -3 tem maior módulo na coluna 2 $\Rightarrow$ troca $L_2 \leftrightarrow L_3$.

Efeito do Pivoteamento — Comparação Numérica

Sem pivoteamento

Dígitos	x_2	Erro x_1
3	0,667	1099%
4	0,6667	100%
5	0,66667	10%
6	0,666667	1%

Com pivoteamento parcial

Dígitos	x_2	Erro x_1
3	0,667	0,1%
4	0,6667	0,01%
5	0,66667	0,001%
6	0,666667	0,0001%

O pivoteamento parcial é **essencial** na prática — sempre utilize!

Pivoteamento Completo

Estratégia

Escolhe como pivô o elemento de **maior módulo em toda a submatriz ativa**:

$$\max_{i,j \geq k} |a_{ij}^{(k-1)}| = |a_{rs}^{(k-1)}|$$

Troca-se a linha $k \leftrightarrow r$ e a coluna $k \leftrightarrow s$.

Atenção

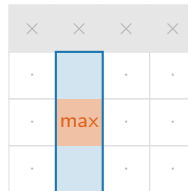
Ao trocar colunas, é necessário **permutar também a ordem das incógnitas** no vetor solução.

Parcial vs. Completo

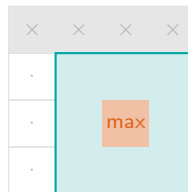
Comparação resumida

	Parcial	Completo
Busca	coluna	submatriz
Troca	linhas	linhas + colunas
Custo	$O(n^2)$	$O(n^3)$
Fator cresc.	$\rho_n \leq 2^{n-1}$	$\rho_n = O(n^{1/4} \log n)$
Uso	comum	raro

Na prática, o **pivoteamento parcial** é suficiente para a maioria dos problemas.



Parcial



Completo

Operações vetoriais

- Escalar $\times$ vetor ax : n flops
- Produto interno $x \cdot y$: $2n - 1$ flops
- Produto Ax (A é $n \times n$): $2n^2 - n$ flops

Produto de matrizes AB ($n \times n$)

Cada entrada d_{ij} custa $2n - 1$ flops. No total:

$$n^2 \times (2n - 1) = 2n^3 - n^2 \text{ flops}$$

Eliminação de Gauss — resumo

- Eliminação progressiva:
 $\frac{n^3}{3}$ somas + $\frac{n^3}{3}$ mult./div.
- Substituição retroativa: n^2 flops

$$\text{Total} \approx \frac{2n^3}{3} + \mathcal{O}(n^2)$$

Observação

$\mathcal{O}(n^m)$ significa “termos de ordem n^m e menores”.

Custo em Números — Crescimento Cúbico

n	#Elim.	#Subst.	#Total	$2n^3/3$ (% elim.)	
10	705	100	805	667	(87,6%)
100	671 550	10 000	681 550	666 667	(98,5%)
1 000	$6,67 \times 10^8$	10^6	$6,67 \times 10^8$	$6,67 \times 10^8$	(99,9%)

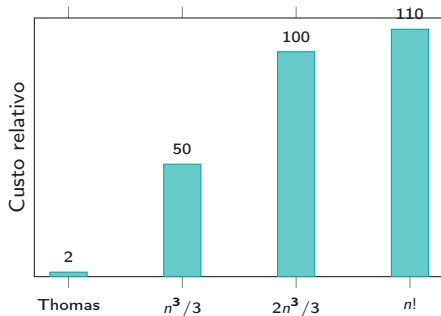
O que isso significa?

A cada vez que n aumenta $10\times$, o número de operações cresce **1000** $\times$ (cúbico!).

Conclusão prática

Quase todo o esforço está na **fase de eliminação**. Melhorias algorítmicas devem focar nessa etapa.

Comparando Abordagens



Resumo dos custos

- **Regra de Cramer:** $\mathcal{O}(n!)$
Inviável para n grande
- **Gauss-Jordan:** $\mathcal{O}(n^3)$
 $\approx 50\%$ mais que Gauss
- **Gauss (ingênuo):** $\mathcal{O}(2n^3/3)$
Referência
- **Algoritmo de Thomas:** $\mathcal{O}(n)$
Apenas para matrizes tridiagonais

Fatoração LU — Motivação

Ideia

Decompor $A = LU$, onde:

- L : matriz **triangular inferior** com diagonal unitária
- U : matriz **triangular superior**

Vantagem

Se b for alterado, a fatoração **não precisa ser refeita** — basta resolver dois sistemas triangulares novamente.

Ideal para múltiplos vetores b com a mesma matriz A .

Como resolver $Ax = b$?

$$Ax = b \Leftrightarrow LUx = b$$

1. Faça $y = Ux$
2. Resolva $Ly = b$ $\rightarrow y$
3. Resolva $Ux = y$ $\rightarrow x$

Cada sistema triangular é resolvido em $O(n^2)$.

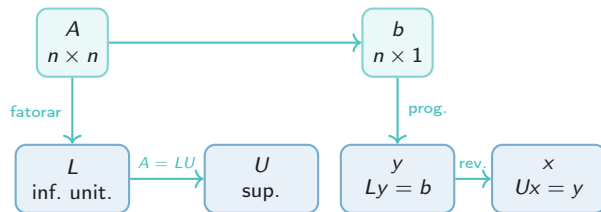
Motivação da Fatoração LU

Problema: múltiplos lados direitos

Mesmo A , diferentes $b^{(1)}, b^{(2)}, \dots, b^{(m)}$.

Gauss repetido: $m \cdot \frac{2n^3}{3}$ flops.

Fatoração LU: fatorar $A = LU$ uma vez ($\frac{2n^3}{3}$) e resolver cada b em $O(n^2)$.



Pipeline de resolução

$$Ax = b \xrightarrow{A=LU} \underbrace{Ly = b}_{\text{subst. prog.}} \quad \text{e} \quad \underbrace{Ux = y}_{\text{subst. rev.}}$$

Exemplo: PageRank

Resolver $(I - \alpha P^T)\pi = \frac{1-\alpha}{n}e$ com $\alpha \approx 0,85$. A fatoração fornece a base teórica dos métodos iterativos.

Como Obter L e U ?

Método de Doolittle (via Gauss)

Os **multiplicadores** m_{ik} calculados na eliminação de Gauss formam exatamente a parte triangular inferior de L :

$$L = \begin{pmatrix} 1 & 0 & \cdots & 0 \\ m_{21} & 1 & \cdots & 0 \\ \vdots & \ddots & \ddots & \vdots \\ m_{n1} & \cdots & m_{n,n-1} & 1 \end{pmatrix}, \quad U = A^{(n-1)} \text{ (triangular superior)}$$

Resumo: L guarda o "histórico" da eliminação

- U : resultado final da eliminação progressiva
- L : multiplicadores usados ao longo do processo
- Ambas cabem na memória de A (armazenamento compacto!)

Estrutura de L e U

$$\begin{array}{cc} L & U \\ \begin{bmatrix} 1 & & 0 & 0 \\ m_{21} & 1 & & 0 \\ m_{31} & m_{32} & 1 & \\ & & & 1 \end{bmatrix} & \begin{bmatrix} u_{11} & & & \\ 0 & u_{22} & & \\ 0 & 0 & u_{33} & \\ & & & u_{44} \end{bmatrix} \end{array}$$

Fatoração PLU

Com pivoteamento: $PA = LU$. P registra as trocas de linha.

Exemplo — Fatoração LU

Sistema

$$3x_1 + 2x_2 + 4x_3 = 1; \quad x_1 + x_2 + 2x_3 = 2; \quad 4x_1 + 3x_2 + 2x_3 = 3$$

Após eliminação:

$$L = \begin{pmatrix} 1 & 0 & 0 \\ \frac{1}{3} & 1 & 0 \\ \frac{4}{3} & 1 & 1 \end{pmatrix}$$

$$U = \begin{pmatrix} 3 & 2 & 4 \\ 0 & \frac{1}{3} & \frac{2}{3} \\ 0 & 0 & -4 \end{pmatrix}$$

Resolvendo $Ly = b$:

$$y_1 = 1, \quad y_2 = \frac{5}{3}, \quad y_3 = 0$$

Resolvendo $Ux = y$:

$$x_3 = 0, \quad x_2 = 5, \quad x_1 = -3$$

Verificação: $LU = A?$ ✓

Verifique multiplicando L e U — o resultado deve ser A .

Custo da Fatoração LU

Decomposição $A = LU$

$$\frac{2n^3}{3} - \frac{n^2}{2} - \frac{n}{6} \approx \frac{2n^3}{3} \text{ flops}$$

Resolução de $Ly = b$ e $Ux = y$

$$2n^2 \text{ flops}$$

$$\text{Total: } \approx \frac{2n^3}{3} + \mathcal{O}(n^2)$$

Para k vetores b : apenas $+ 2kn^2$ flops adicionais — a fatoração é reaproveitada!

Implementação em Python: LU e PLU

```
import numpy as np
from scipy.linalg import lu

def fatoracao_lu(A):
    """Doolittle: A = LU sem pivoteamento."""
    A = np.array(A, dtype=float)
    n = A.shape[0]
    L, U = np.eye(n), np.zeros((n, n))
    for k in range(n):
        for j in range(k, n):
            U[k,j] = A[k,j] - L[k,:k] @ U[:k,j]
        for i in range(k+1, n):
            L[i,k] = (A[i,k] - L[i,:k] @ U[:k,k]) / U[k,k]
    return L, U

def resolver_lu(L, U, b):
    """Resolve Ax=b dado A=LU."""
    n = len(b)
    y = np.zeros(n)
    for i in range(n):
        y[i] = b[i] - L[i,:i] @ y[:i]
    x = np.zeros(n)
    for i in range(n-1,-1,-1):
        x[i] = (y[i] - U[i,i+1:] @ x[i+1:]) / U[i,i]
    return x

A = np.array([[3,2,4],[1,1,2],[4,3,2]], dtype=float)
b = np.array([1, 2, 3], dtype=float)
L, U = fatoracao_lu(A)
print("Solucao:", resolver_lu(L, U, b))
```

[-3. 5. 0.]

Fatoração de Cholesky — Matrizes SPD

Matriz SPD

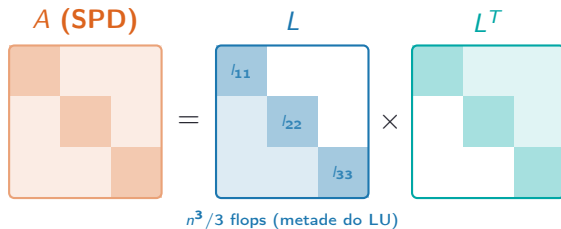
A simétrica é *positiva definida* (SPD) se $x^T A x > 0$ para todo $x \neq 0$.

Theorem (Cholesky)

Se A é SPD, existe único L triangular inferior com diagonal positiva tal que $A = LL^T$.

Vantagens sobre LU

- **Custo:** $n^3/3$ flops (metade do LU!).
- **Sem pivoteamento** necessário.
- **Mais estável** numericamente.
- Se $l_{kk}^2 < 0$: A **não** é SPD.



Aplicações

Equações normais $(X^T X)\beta = X^T y$;
Processos Gaussianos (K SPD por construção).

Cholesky — Fórmulas de Cálculo

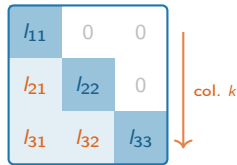
Como calcular os elementos de L

Percorrendo coluna por coluna ($k = 1, 2, \dots, n$):

$$l_{kk} = \sqrt{a_{kk} - \sum_{j=1}^{k-1} l_{kj}^2}, \quad l_{ik} = \frac{1}{l_{kk}} \left(a_{ik} - \sum_{j=1}^{k-1} l_{ij} l_{kj} \right), \quad i > k.$$

Pipeline de resolução: $Ax = b$

1. Fatorar $A = LL^T$ custo $n^3/3$
2. Resolver $Ly = b$ subst. progressiva
3. Resolver $L^T x = y$ subst. retroativa



Atenção

Se em algum passo $a_{kk} - \sum l_{kj}^2 \leq 0$, a matriz **não é SPD** e o algoritmo para.

Exemplo — Fatoração de Cholesky (Passo a Passo)

Sistema

$$A = \begin{pmatrix} 4 & 2 & 2 \\ 2 & 3 & 0 \\ 2 & 0 & 3 \end{pmatrix}, \quad b = \begin{pmatrix} 8 \\ 5 \\ 5 \end{pmatrix} \quad (\text{verifique: } A \text{ é SPD — diagonal positiva e } \det > 0)$$

Calculando L coluna a coluna:

$$k = 1: \quad l_{11} = \sqrt{4} = 2$$

$$l_{21} = \frac{2}{2} = 1, \quad l_{31} = \frac{2}{2} = 1$$

$$k = 2: \quad l_{22} = \sqrt{3 - 1^2} = \sqrt{2} \approx 1,414$$

$$l_{32} = \frac{0 - 1 \cdot 1}{\sqrt{2}} = \frac{-1}{\sqrt{2}} \approx -0,707$$

$$k = 3: \quad l_{33} = \sqrt{3 - 1^2 - (-\frac{1}{\sqrt{2}})^2} = \sqrt{3 - 1 - \frac{1}{2}} = \sqrt{\frac{3}{2}} \approx 1,225$$

Resultado:

$$L = \begin{pmatrix} 2 & 0 & 0 \\ 1 & \sqrt{2} & 0 \\ 1 & -\frac{1}{\sqrt{2}} & \sqrt{\frac{3}{2}} \end{pmatrix}$$

Verificação: $LL^T = A?$ ✓

Multiplique $L \cdot L^T$ e confirme que o resultado é A .

Exemplo — Resolução via Cholesky

Continuação: resolver $Ax = b$ com $A = LL^T$, $b = (8, 5, 5)^T$

Passo 1: Resolver $Ly = b$ (subst. progressiva)

$$\begin{pmatrix} 2 & 0 & 0 \\ 1 & \sqrt{2} & 0 \\ 1 & -\frac{1}{\sqrt{2}} & \sqrt{\frac{3}{2}} \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix} = \begin{pmatrix} 8 \\ 5 \\ 5 \end{pmatrix}$$

$$y_1 = \frac{8}{2} = 4$$

$$y_2 = \frac{5 - 1 \cdot 4}{\sqrt{2}} = \frac{1}{\sqrt{2}} \approx 0,707$$

$$y_3 = \frac{5 - 1 \cdot 4 - (-\frac{1}{\sqrt{2}}) \cdot \frac{1}{\sqrt{2}}}{\sqrt{\frac{3}{2}}} = \frac{5 - 4 + \frac{1}{2}}{\sqrt{\frac{3}{2}}} = \frac{\frac{3}{2}}{\sqrt{\frac{3}{2}}} \approx 1,225$$

Passo 2: Resolver $L^T x = y$ (subst. retroativa)

$$\begin{pmatrix} 2 & 1 & 1 \\ 0 & \sqrt{2} & -\frac{1}{\sqrt{2}} \\ 0 & 0 & \sqrt{\frac{3}{2}} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 4 \\ \frac{1}{\sqrt{2}} \\ \frac{3/2}{\sqrt{3/2}} \end{pmatrix}$$

$$x_3 = \frac{y_3}{\sqrt{3/2}} = 1$$

$$x_2 = \frac{y_2 - (-\frac{1}{\sqrt{2}}) \cdot 1}{\sqrt{2}} = \frac{\frac{1}{\sqrt{2}} + \frac{1}{\sqrt{2}}}{\sqrt{2}} = 1$$

$$x_1 = \frac{4 - 1 \cdot 1 - 1 \cdot 1}{2} = 1$$

Algoritmo de Thomas — Matrizes Tridiagonais

Sistema tridiagonal

$$\begin{bmatrix} b_1 & c_1 & & & & \\ a_2 & b_2 & c_2 & & & \\ & \ddots & \ddots & \ddots & & \\ & & & a_n & b_n & \end{bmatrix} x = d$$

Aparece na discretização de EDPs, splines, circuitos em malha.

Matriz tridiagonal 6×6

b	c	0	0	0	0
a	b	c	0	0	0
0	a	b	c	0	0
0	0	a	b	c	0
0	0	0	a	b	c
0	0	0	0	a	b

- diagonal (b_i)
- sub/sup (a_i, c_i)
- zeros

Theorem (Thomas = LU especializado)

Um sistema tridiagonal pode ser resolvido em $O(n)$ operações (contra $O(n^3)$ do Gauss geral).

Economia de memória

Armazena apenas $3n - 2$ elementos (não n^2).

Algoritmo de Thomas — Duas Etapas

Etapa 1: Eliminação progressiva

Para $k = 2, 3, \dots, n$:

$$m_k = \frac{a_k}{b_{k-1}}, \quad b_k \leftarrow b_k - m_k c_{k-1}, \\ d_k \leftarrow d_k - m_k d_{k-1}$$

Zera a subdiagonal, transformando o sistema em triangular superior.

Etapa 2: Substituição retroativa

$$x_n = \frac{d_n}{b_n}, \quad x_k = \frac{d_k - c_k x_{k+1}}{b_k},$$

para $k = n-1, \dots, 1$.

Início

b_1	c_1	0	0
a_2	b_2	c_2	0
0	a_3	b_3	c_3
0	0	a_4	b_4

elim. $\rightarrow$

Triangular $\uparrow$

b'_1	c_1		
0	b'_2	c_2	
0	0	b'_3	c_3
0	0	0	b'_4

Custo total

$5(n-1)$ operações na eliminação
+ $3(n-1) + 1$ na substituição = $O(n)$.

Exemplo Guiado — Algoritmo de Thomas

Sistema tridiagonal 4×4

$$\begin{pmatrix} 4 & 1 & 0 & 0 \\ -2 & 6 & -3 & 0 \\ 0 & 4 & 5 & -1 \\ 0 & 0 & -3 & 7 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} 5 \\ 3 \\ 0 \\ 6 \end{pmatrix}$$

Identificando as diagonais:

Vetores de entrada

- Diagonal principal: $b = (4, 6, 5, 7)$
- Subdiagonal: $a = (-2, 4, -3)$ (linhas 2, 3, 4)
- Superdiagonal: $c = (1, -3, -1)$ (linhas 1, 2, 3)
- Lado direito: $d = (5, 3, 0, 6)$

Roteiro do método

E1. Eliminação progressiva

Para $k = 2, 3, 4$:

$$m_k = a_k / b'_{k-1}$$

$$b'_k = b_k - m_k c_{k-1}$$

$$d'_k = d_k - m_k d'_{k-1}$$

E2. Substituição retroativa

De x_4 até x_1

Etapa 1 — Eliminação Progressiva (passos $k = 2$ e $k = 3$)

Fórmulas

$$m_k = \frac{a_k}{b'_{k-1}}, \quad b'_k = b_k - m_k \cdot c_{k-1}, \quad d'_k = d_k - m_k \cdot d'_{k-1}$$

Passo $k = 2$ — zerando $a_2 = -2$

$$m_2 = \frac{a_2}{b'_1} = \frac{-2}{4} = -\frac{1}{2}$$

$$b'_2 = b_2 - m_2 \cdot c_1 = 6 - \left(-\frac{1}{2}\right)(1) = 6 + \frac{1}{2} = \frac{13}{2}$$

$$d'_2 = d_2 - m_2 \cdot d'_1 = 3 - \left(-\frac{1}{2}\right)(5) = 3 + \frac{5}{2} = \frac{11}{2}$$

Passo $k = 3$ — zerando $a_3 = 4$

$$m_3 = \frac{a_3}{b'_2} = \frac{4}{13/2} = \frac{8}{13}$$

Etapa 1 — Eliminação Progressiva (passo $k = 4$) e Resumo

Passo $k = 4$ — zerando $a_4 = -3$

$$m_4 = \frac{a_4}{b'_3} = \frac{-3}{89/13} = -\frac{39}{89}$$

$$b'_4 = b_4 - m_4 \cdot c_3 = 7 - \left(-\frac{39}{89}\right)(-1) = 7 - \frac{39}{89} = \frac{623 - 39}{89} = \frac{584}{89}$$

$$d'_4 = d_4 - m_4 \cdot d'_3 = 6 - \left(-\frac{39}{89}\right)\left(-\frac{44}{13}\right) = 6 - \frac{39 \cdot 44}{89 \cdot 13} = 6 - \frac{132}{89} = \frac{534 - 132}{89} = \frac{402}{89}$$

Resumo após eliminação progressiva

k	m_k	b'_k	c_k	d'_k
1	—	4	1	5
2	$-1/2$	$13/2$	-3	$11/2$
3	$8/13$	$89/13$	-1	$-44/13$
4	$-39/89$	$584/89$	—	$402/89$

Etapa 2 — Substituição Retroativa e Verificação

Fórmulas

$$x_n = \frac{d'_n}{b'_n}, \quad x_k = \frac{d'_k - c_k x_{k+1}}{b'_k}, \quad k = n-1, \dots, 1$$

Cálculo (de baixo para cima)

$$x_4 = \frac{d'_4}{b'_4} = \frac{402/89}{584/89} = \frac{402}{584} = \frac{201}{292} \approx 0,6884$$

$$x_3 = \frac{d'_3 - c_3 x_4}{b'_3} = \frac{-\frac{44}{13} - (-1)\frac{201}{292}}{\frac{89}{13}} = \frac{-\frac{1248}{292} + \frac{201}{292}}{\frac{89}{13}} = \frac{-\frac{1047}{292}}{\frac{89}{13}} = -\frac{1047 \cdot 13}{292 \cdot 89} = -\frac{1047}{2252} \approx -0,3939$$

$$x_2 = \frac{d'_2 - c_2 x_3}{b'_2} = \frac{\frac{11}{2} - (-3)\left(-\frac{1047}{2252}\right)}{\frac{13}{2}} = \frac{\frac{11}{2} - \frac{3141}{2252}}{\frac{13}{2}} = \frac{\frac{12386 - 3141}{2252}}{\frac{13}{2}} = \frac{9245}{2252} \cdot \frac{2}{13} = \frac{9245}{14638} \approx 0,6319$$

$$x_1 = \frac{d'_1 - c_1 x_2}{b'_1} = \frac{5 - (1)\frac{9245}{14638}}{4} = \frac{\frac{73190 - 9245}{14638}}{4} = \frac{63945}{58552} \approx 1,0921$$

Solução aproximada

$$x \approx \begin{pmatrix} 1,092 \\ 0,632 \\ -0,394 \\ 0,688 \end{pmatrix}$$

Verificação: $Ax \stackrel{?}{=} d$

$$4(1,092) + 1(0,632) \approx 5,00 \checkmark$$

$$-2(1,092) + 6(0,632) - 3(-0,394) \approx 3,00 \checkmark$$

$$4(0,632) + 5(-0,394) - 1(0,688) \approx 0,00 \checkmark$$

$$-3(-0,394) + 7(0,688) \approx 6,00 \checkmark$$

Exemplo Guiado — Algoritmo de Thomas

Sistema tridiagonal 4×4

$$\begin{pmatrix} 4 & -1 & 0 & 0 \\ -2 & 6 & -3 & 0 \\ 0 & -1 & 5 & -2 \\ 0 & 0 & -3 & 7 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} 2 \\ 10 \\ -3 \\ 6 \end{pmatrix}$$

Identificando as diagonais:

Vetores de entrada

- Diagonal principal: $b = (4, 6, 5, 7)$
- Subdiagonal: $a = (-2, -1, -3)$ (linhas 2, 3, 4)
- Superdiagonal: $c = (-1, -3, -2)$ (linhas 1, 2, 3)
- Lado direito: $d = (2, 10, -3, 6)$

Roteiro do método

- E1.** Eliminação progressiva
Para $k = 2, 3, 4$: calcular m_k ,
atualizar b'_k e d'_k
- E2.** Substituição retroativa
De x_4 até x_1

Etapa 1 — Eliminação Progressiva (passos $k = 2$ e $k = 3$)

Fórmulas aplicadas a cada passo

$$m_k = \frac{a_k}{b'_{k-1}}, \quad b'_k = b_k - m_k \cdot c_{k-1}, \quad d'_k = d_k - m_k \cdot d'_{k-1}$$

Passo $k = 2$ — zerando $a_2 = -2$

$$m_2 = \frac{a_2}{b'_1} = \frac{-2}{4} = -0,5$$

$$b'_2 = b_2 - m_2 \cdot c_1 = 6 - (-0,5)(-1) = 6 - 0,5 = 5,5$$

$$d'_2 = d_2 - m_2 \cdot d'_1 = 10 - (-0,5)(2) = 10 + 1 = 11$$

Passo $k = 3$ — zerando $a_3 = -1$

$$m_3 = \frac{a_3}{b'_2} = \frac{-1}{5,5} = -\frac{2}{11} \approx -0,18\overline{1}$$

Etapa 1 — Eliminação Progressiva (passo $k = 4$) e Resumo

Passo $k = 4$ — zerando $a_4 = -3$

$$m_4 = \frac{a_4}{b'_3} = \frac{-3}{49/11} = -\frac{33}{49}$$

$$b'_4 = b_4 - m_4 \cdot c_3 = 7 - \left(-\frac{33}{49}\right)(-2) = 7 - \frac{66}{49} = \frac{343 - 66}{49} = \frac{277}{49}$$

$$d'_4 = d_4 - m_4 \cdot d'_3 = 6 - \left(-\frac{33}{49}\right)(-1) = 6 - \frac{33}{49} = \frac{294 - 33}{49} = \frac{261}{49}$$

Resumo após eliminação progressiva

k	m_k	b'_k	c_k	d'_k
1	—	4	-1	2
2	-1/2	11/2	-3	11
3	-2/11	49/11	-2	-1
4	-33/49	277/49	—	261/49

Etapa 2 — Substituição Retroativa e Verificação

Fórmulas

$$x_n = \frac{d'_n}{b'_n}, \quad x_k = \frac{d'_k - c_k x_{k+1}}{b'_k}, \quad k = n-1, \dots, 1$$

Cálculo (de baixo para cima)

$$x_4 = \frac{d'_4}{b'_4} = \frac{261/49}{277/49} = \frac{261}{277} \approx 0,942$$

$$x_3 = \frac{d'_3 - c_3 x_4}{b'_3} = \frac{-1 - (-2)(261/277)}{49/11} = \frac{-1 + 522/277}{49/11} = \frac{245/277}{49/11} = \frac{245 \cdot 11}{277 \cdot 49} \frac{2695}{13573} = \frac{2695}{277 \cdot 49}$$

$$x_2 = \frac{d'_2 - c_2 x_3}{b'_2} = \frac{11 - (-3)(55/277)}{11/2} = \frac{11 + 165/277}{11/2} = \frac{(3047 + 165)/277}{11/2} = \frac{3212}{277} \cdot \frac{2}{11} = \frac{6424}{3047}$$

$$x_1 = \frac{d'_1 - c_1 x_2}{b'_1} = \frac{2 - (-1)(6424/3047)}{1}$$

Solução aproximada

$$x \approx \begin{pmatrix} 1,027 \\ 2,108 \\ 0,199 \\ 0,942 \end{pmatrix}$$

Verificação: $Ax \stackrel{?}{=} d$

$$4(1,027) - 1(2,108) \approx 2,00 \checkmark$$

$$-2(1,027) + 6(2,108) - 3(0,199) \approx 10,0 \checkmark$$

$$-1(2,108) + 5(0,199) - 2(0,942) \approx -3,00 \checkmark$$

$$-3(0,199) + 7(0,942) \approx 6,00 \checkmark$$

Número de Condição e Sensibilidade

Definição

$$\kappa(A) = \|A\| \cdot \|A^{-1}\|.$$

Para norma espectral: $\kappa_2(A) = \Lambda_{\max}/\lambda_{\min}$.

Theorem (Amplificação de erros)

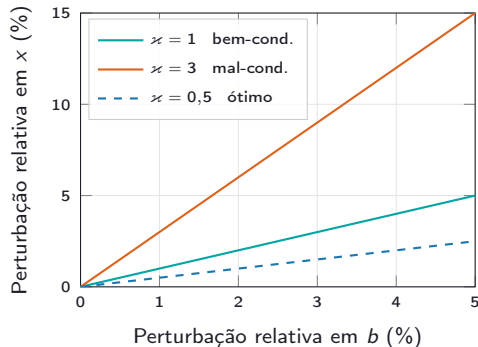
Se $Ax = b$ e $A(x + \delta x) = b + \delta b$:

$$\frac{\|\delta x\|}{\|x\|} \leq \kappa(A) \frac{\|\delta b\|}{\|b\|}.$$

Regra prática

Se $\kappa(A) \approx 10^k$ e aritmética tem d dígitos, a solução tem **no máximo** $d - k$ dígitos corretos.

Ex.: $\kappa(A) \approx 10^{10}$, double ($d = 16$) $\Rightarrow$ apenas 6 dígitos.



Matriz de Hilbert (mal-cond.)

$$h_{ij} = \frac{1}{i+j-1}. \quad \kappa_2(H_8) \approx 10^{10}.$$

Fluxograma de Escolha do Método

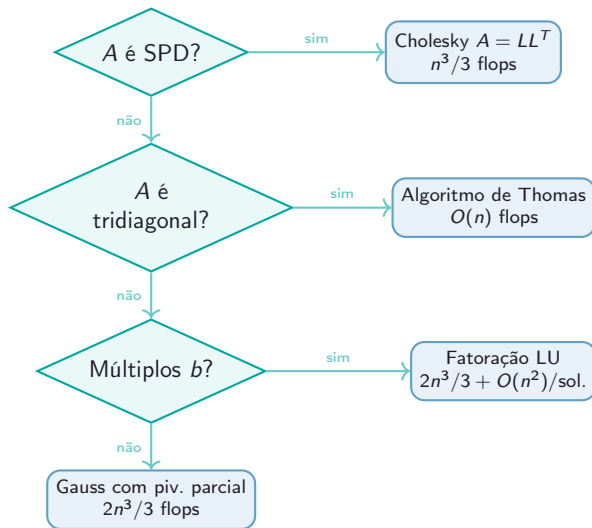
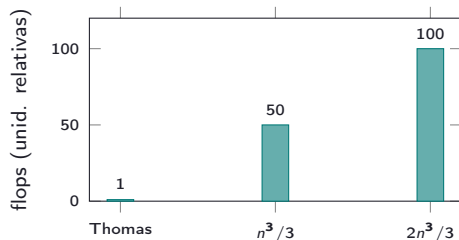


Tabela Comparativa

Método	Flops	Mem.	Pivoteamento	Quando usar
Gauss (sem piv.)	$2n^3/3$	$O(n^2)$	—	Apenas educacional
Gauss (com piv.)	$2n^3/3$	$O(n^2)$	Parcial	Geral, único b
Fatoração LU	$2n^3/3$	$O(n^2)$	Parcial	Múltiplos b
Cholesky	$n^3/3$	$O(n^2)$	Não	A SPD (otim., ML)
Thomas	$O(n)$	$O(n)$	Não	Tridiagonal (EDPs)



Recomendação prática

Use `numpy.linalg.solve` ou `scipy.linalg.lu_solve` — implementam PLU com LAPACK e detectam automaticamente estrutura.

Exercícios Propostos

Ex. 1 — Gauss Manual

Resolva por eliminação de Gauss, mostrando cada etapa da matriz aumentada:

$$\begin{cases} 3x_1 + 2x_2 + 4x_3 = 1 \\ x_1 + x_2 + 2x_3 = 2 \\ 4x_1 + 3x_2 - 2x_3 = 3 \end{cases}.$$

Ex. 2 — Pivoteamento

Para $\begin{bmatrix} 0 & 1 \\ 2 & 3 \end{bmatrix} x = \begin{bmatrix} 1 \\ 5 \end{bmatrix}$:

- (a) mostre que sem pivoteamento o método falha;
- (b) aplique pivoteamento parcial e resolva;
- (c) calcule o fator de crescimento ρ_2 .

Fundamentos

- Problema: $Ax = b$, $A \in \mathbb{R}^{m \times n}$
- Solução única $\Leftrightarrow \det(A) \neq 0$
- 3 casos: única, infinitas ou nenhuma
- Diretos $\mathcal{O}(2n^3/3)$ vs. iterativos $\mathcal{O}(n^2)/\text{iter.}$

Condicionalmento

- $\kappa(A) = \|A\| \cdot \|A^{-1}\|$
- $\kappa \approx 10^k \Rightarrow$ perde k dígitos
- Hilbert H_n : exemplo mal-condicionado
- Nunca use $\det(A)$ para singularidade!

Eliminação de Gauss

- Triangularização de $[A \mid b]$
- Multiplicador: $m_{ik} = a_{ik}/a_{kk}$
- Substituição retroativa
- Custo: $\approx 2n^3/3$ flops

Pivoteamento

- Parcial: busca na coluna — recomendado
- Completo: busca na submatriz — raro
- Evita divisão por zero e instabilidade

Fatoração LU

- $A = LU$ (Doolittle); com piv.: $PA = LU$
- L guarda os multiplicadores de Gauss
- Ideal para múltiplos vetores b
- $Ly = b$ e $Ux = y$: cada um em $\mathcal{O}(n^2)$

Quando usar cada método

- **SPD** $\rightarrow$ Cholesky ($n^3/3$)
- **Tridiagonal** $\rightarrow$ Thomas ($\mathcal{O}(n)$)
- **Múltiplos b** $\rightarrow$ Fatoração LU
- **Geral** $\rightarrow$ Gauss com piv. parcial

Prática: `numpy.linalg.solve` ou `scipy.linalg.lu_solve` — PLU via LAPACK

- **Trefethen & Bau** (1997). *Numerical Linear Algebra*. SIAM. — álgebra linear numérica com foco em estabilidade.
- **Golub & Van Loan** (2013). *Matrix Computations*. 4.^a ed. JHU Press. — enciclopédia de computações matriciais.
- **Burden & Faires** (2015). *Numerical Analysis*. 10.^a ed. Cengage. Cap. 6: eliminação gaussiana e LU.
- **Franco** (2007); **Ruggiero & Lopes** (1997) — referências brasileiras com exemplos detalhados.
- **LAPACK**: <http://www.netlib.org/lapack/>
- **NumPy linalg**: <https://numpy.org/doc/stable/reference/routines.linalg.html>
- **SciPy linalg**: <https://docs.scipy.org/doc/scipy/reference/linalg.html>